```
# ============================================================
# STEP 0: Install Required Libraries
# ============================================================
!pip install -q faiss-cpu transformers torch pandas numpy

# ============================================================
# STEP 1: Upload Dataset (MovieLens / Amazon Product CSV)
# ============================================================
from google.colab import files

uploaded = files.upload()

# ============================================================
# STEP 2: Load Dataset
# ============================================================
import pandas as pd

file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)

print("Dataset loaded successfully")
print(df.head())

# ============================================================
# STEP 3: Select Text Column for Embedding
# (First column used by default)
# ============================================================
TEXT_COLUMN = df.columns[0]

item_texts = df[TEXT_COLUMN].astype(str).tolist()

print("\nTotal items:", len(item_texts))
print("Sample item:", item_texts[0])

# ============================================================
# STEP 4: Load Transformer Model
# ============================================================
from transformers import AutoTokenizer, AutoModel
import torch
import numpy as np
import faiss

device = "cuda" if torch.cuda.is_available() else "cpu"

model_name = "sentence-transformers/all-MiniLM-L6-v2"

tokenizer = AutoTokenizer.from_pretrained(model_name)
encoder = AutoModel.from_pretrained(model_name).to(device)

encoder.eval()
```

```python
# ============================================================
# STEP 5: Encode Texts into High-Dimensional Embeddings
# ============================================================
def encode_texts(texts, batch_size=16):

    all_embeddings = []

    with torch.no_grad():

        for i in range(0, len(texts), batch_size):

            batch = texts[i:i+batch_size]

            tokens = tokenizer(
                batch,
                padding=True,
                truncation=True,
                return_tensors="pt"
            ).to(device)

            outputs = encoder(**tokens)

            embeddings = outputs.last_hidden_state.mean(dim=1)

            embeddings = embeddings.cpu().numpy().astype("float32")

            all_embeddings.append(embeddings)

    return np.vstack(all_embeddings)


item_embeddings = encode_texts(item_texts)

# Normalize for cosine similarity
faiss.normalize_L2(item_embeddings)

print("\nEmbeddings shape:", item_embeddings.shape)

# ============================================================
# STEP 6: Build FAISS Index
# ============================================================
embedding_dim = item_embeddings.shape[1]

index = faiss.IndexFlatIP(embedding_dim)

index.add(item_embeddings)

print("FAISS index built with", index.ntotal, "items")

# ============================================================
# STEP 7: Similarity Search Function
```

```python
# ============================================================
def get_similar_items(item_id, k=5):

    query_vector = item_embeddings[item_id:item_id+1]

    scores, indices = index.search(query_vector, k)

    return scores[0], indices[0]

# ============================================================
# STEP 8: Test Similarity Retrieval
# ============================================================
query_id = 0   # change index if needed

scores, idxs = get_similar_items(query_id, k=5)

print("\nQUERY ITEM:")
print(item_texts[query_id])

print("\nSIMILAR ITEMS:")

for score, idx in zip(scores, idxs):

    print(f"-> {item_texts[idx]}  (score={score:.3f})")
```

```
<IPython.core.display.HTML object>

Saving movie.csv to movie.csv
Dataset loaded successfully
   movieId                                title  \
0        1                    Toy Story (1995)
1        2                      Jumanji (1995)
2        3             Grumpier Old Men (1995)
3        4            Waiting to Exhale (1995)
4        5  Father of the Bride Part II (1995)

                                        genres
0  Adventure|Animation|Children|Comedy|Fantasy
1                   Adventure|Children|Fantasy
2                               Comedy|Romance
3                        Comedy|Drama|Romance
4                                       Comedy

Total items: 27278
Sample item: 1

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
```

```
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```


```
{"model_id":"a804f553f1d4434daf250b4a771030ed","version_major":2,"vers
ion_minor":0}

{"model_id":"cd23db03f9584b3293b2f53b29770287","version_major":2,"vers
ion_minor":0}

{"model_id":"da754285177e4143989d39a2609d7948","version_major":2,"vers
ion_minor":0}

{"model_id":"3d50bebb13a64c478c4f7530f920e9cf","version_major":2,"vers
ion_minor":0}

{"model_id":"6f42952e762c446bbd29f4535a7e0d6a","version_major":2,"vers
ion_minor":0}

{"model_id":"2786ababd27c4757bd9447331bcc176f","version_major":2,"vers
ion_minor":0}

{"model_id":"776d5a2667b04426b21dc4bf62212b70","version_major":2,"vers
ion_minor":0}
```


```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.


Embeddings shape: (27278, 384)
FAISS index built with 27278 items

QUERY ITEM:
1

SIMILAR ITEMS:
-> 1  (score=1.000)
-> 2  (score=0.805)
```

```
-> 3  (score=0.791)
-> 4  (score=0.769)
-> 7  (score=0.658)
```